



UNIVERSIDAD
TECNOLOGICA DE
PANAMA
FACULTAD DE INGENIERÍA
DE SISTEMAS
COMPUTACIONALES



PLAN DEL PROYECTO SEMESTRAL EVENFLOW

MATERIA:
DESARROLLO DE SOFTW. PARA PLATAF. MÓVIL.

PROFESOR:
JORGE CISNEROS

INTEGRANTES
Félix Beitia 8-1014-968
Carlos Delgado E-8-204947
Cristopher Quintero 8-1015-1972
Eriol Tuñon 8-1017-293
Andres Vega 8-1003-1210

GRUPO:
ISF-241

I SEMESTRE 2026

ÍNDICE DE CONTENIDO

Resumen Ejecutivo	8
Introducción	9
Acta de Constitución del Proyecto	10
Objetivo del Proyecto	10
Objetivos específicos	11
Justificación.....	11
Stakeholders	12
Restricciones	12
Restricciones tecnológicas.....	12
Restricciones académicas	12
Restricciones funcionales.....	13
Restricciones de seguridad	13
Supuestos	13
Presupuesto Académico	14
Recursos de software	14
Recursos de hardware	14
Recursos humanos	14
Estimación del esfuerzo.....	14
Plan del Proyecto.....	15
Planificación General	15
Cronograma del Proyecto.....	16
Entregables	16
Entregables de planificación.....	16
Entregables de desarrollo.....	16
Entregables técnicos	17
Entregables finales	17
Hitos del Proyecto.....	17
Roadmap del Proyecto	18
Módulo 1 – Gestión de Usuarios y Seguridad	18

Módulo 2 – Gestión de Eventos y Venta de Boletos	18
Módulo 3 – Gestión de Estacionamientos	19
Módulo 4 – QR Dinámicos y Check-in	19
Módulo 5 – Recuperación y Reembolsos	19
Dependencias entre módulos	20
Alcance del Proyecto	20
Alcance General	20
Alcance Funcional	21
Alcance Técnico	21
Exclusiones del Alcance	22
Criterios de Finalización	22
Requisitos del Sistema	23
Requisitos Funcionales	23
RF-01 Gestión de Usuarios	23
RF-02 Autenticación y Autorización	23
RF-03 Gestión de Eventos	24
RF-04 Gestión de Boletos	24
RF-05 Compra de Boletos	24
RF-06 Gestión de Pagos	25
RF-07 Gestión de QR Dinámico	25
RF-08 Check-In	25
RF-09 Gestión de Estacionamientos	26
RF-10 Reembolsos (Refund)	26
RF-11 Intercambio Oficial (Exchange)	26
RF-12 Recovery	27
RF-13 Gestión de Inventario	27
RF-14 Notificaciones	27
RF-15 Auditoría	27
Requisitos No Funcionales	28
Seguridad	28

Rendimiento	28
Disponibilidad	28
Escalabilidad	28
Mantenibilidad	29
Portabilidad	29
Usabilidad	29
Arquitectura del Sistema	29
Arquitectura General	29
Clean Architecture.....	30
Domain-Driven Design.....	30
Tecnologías.....	31
Arquitectura Física	31
Cliente.....	32
Servidor	32
Base de Datos	32
Flujo General de Comunicación	33
Arquitectura Lógica	34
Capas	34
Domain Layer	34
Application Layer	35
Infrastructure Layer	35
Presentation Layer	36
Flujo General	36
Entidades principales	37
Relación General	38
Índices	38
OpenAPI.....	39
Autenticación	39
Usuarios	39
Eventos	39

Órdenes.....	39
Tickets.....	39
Check-In	40
Recovery	40
Payments.....	40
Flujo REST	41
Autenticación	41
Roles	42
Control de acceso	42
QR Dinámico	42
Flujo de autenticación.....	43
Diseño Android.....	43
Arquitectura	43
Jetpack Compose.....	43
Navigation	44
ViewModel.....	44
Repository Pattern	44
Offline First.....	45
Room Database	45
Flujo Android.....	46
Gestión del Riesgo	46
Matriz de Riesgos.....	46
Evaluación de Probabilidad.....	47
Evaluación de Impacto	48
Estrategia de Mitigación.....	48
Gestión del Cambio	48
Architectural Decision Records (ADR)	49
Congelamiento de Arquitectura.....	49
Control de Cambios.....	50
Casos de Uso.....	51

Actores	51
Casos de uso del Visitante	51
Casos de uso del Asistente	51
Casos de uso del Organizador	52
Casos de uso del Staff.....	52
Casos de uso del Administrador	52
Diagrama de Casos de Uso.....	53
Diagramas UML	53
Casos de Uso	53
Diagrama de Componentes	54
Diagrama de Despliegue.....	55
Diagrama de Secuencia (Compra)	55
Manual Técnico.....	56
Introducción	56
Requisitos del sistema	56
Arquitectura del sistema	56
Tecnologías utilizadas.....	57
Backend	57
Aplicación Android.....	57
Base de datos	57
Estructura general del proyecto	58
Configuración de la base de datos	58
Configuración del Backend	58
Configuración de la aplicación Android	59
Ejecución del sistema	59
Consideraciones técnicas	59
Manual de Usuario	60
Introducción	60
Inicio de sesión	60
Registro de usuario	60

Consulta de eventos	60
Consulta del detalle del evento	61
Compra de boletos	62
Consulta de boletos	63
Código QR.....	64
Consulta de órdenes	65
Actualización automática mediante acelerómetro.....	66
Actualización del perfil.....	67
Cierre de sesión	68
Conclusiones	68

Resumen Ejecutivo

EventFlow es una plataforma integral para la gestión de eventos desarrollada como proyecto semestral de la Licenciatura en Ingeniería de Software. El sistema surge como respuesta a diversas problemáticas presentes en la administración de eventos, entre ellas la venta de boletos, el control de acceso, la administración de estacionamientos, la gestión de reembolsos y la seguridad en la validación de entradas.

En muchos eventos, los organizadores enfrentan inconvenientes relacionados con la falsificación de boletos mediante capturas de pantalla o códigos QR estáticos, la dificultad para controlar el acceso de los asistentes en tiempo real, la falta de mecanismos seguros para el intercambio o recuperación de boletos y la limitada integración entre los diferentes procesos administrativos. Estas situaciones afectan tanto la experiencia de los asistentes como la eficiencia operativa de los organizadores.

Con el propósito de atender estas necesidades, se desarrolló **EventFlow**, una solución tecnológica compuesta por una aplicación móvil Android desarrollada con **Kotlin** y **Jetpack Compose**, junto con un backend implementado en **Spring Boot**, siguiendo una arquitectura basada en **Clean Architecture**, principios **SOLID**, patrones de diseño y una estricta separación de responsabilidades entre las diferentes capas del sistema. La persistencia de datos se realiza sobre una base de datos **PostgreSQL**, mientras que la autenticación y autorización se gestionan mediante **JSON Web Tokens (JWT)**.

Durante el desarrollo se adoptó un enfoque modular e incremental, permitiendo construir el sistema mediante entregas sucesivas que facilitaron la validación continua de la arquitectura y de las reglas de negocio. Hasta la fecha de elaboración de este documento, el proyecto contempla la implementación de cinco módulos funcionales que abarcan la autenticación de usuarios, administración de eventos, venta de boletos, generación y validación de códigos QR dinámicos para el proceso de check-in, administración de estacionamientos, así como el proceso de recuperación y reembolso de boletos.

El sistema incorpora múltiples mecanismos de seguridad orientados a garantizar la integridad de la información y prevenir fraudes. Entre ellos destacan la utilización de códigos QR dinámicos con expiración configurable, control de acceso basado en roles, validaciones de concurrencia para evitar operaciones duplicadas, manejo de

idempotencia en operaciones críticas y una arquitectura preparada para futuras integraciones con proveedores reales de pago.

Desde el punto de vista de la ingeniería de software, el proyecto fue diseñado considerando criterios de mantenibilidad, escalabilidad y extensibilidad. Para ello se documentaron las decisiones arquitectónicas mediante **Architecture Decision Records (ADR)**, se definieron contratos de comunicación mediante **OpenAPI**, se establecieron eventos de dominio, diagramas de arquitectura y una estructura modular que facilita la incorporación de nuevas funcionalidades sin afectar los componentes existentes.

Como resultado, **EventFlow** constituye una plataforma robusta para la gestión integral de eventos, preparada para evolucionar hacia futuras funcionalidades como el intercambio oficial de boletos, integración con pasarelas de pago reales, sistemas avanzados de notificaciones y herramientas analíticas para organizadores. El desarrollo alcanzado demuestra la aplicación práctica de metodologías modernas de desarrollo de software, buenas prácticas de arquitectura empresarial y tecnologías ampliamente utilizadas en la industria, proporcionando una solución alineada con los objetivos académicos y con escenarios reales de implementación.

Introducción

La transformación digital ha impulsado la necesidad de desarrollar plataformas capaces de gestionar de manera eficiente los diferentes procesos involucrados en la organización y administración de eventos. En la actualidad, muchas soluciones existentes ofrecen únicamente funcionalidades básicas para la venta de boletos, dejando de lado aspectos importantes como la gestión integral del ciclo de vida del ticket, el control de acceso mediante códigos QR dinámicos, la administración de

estacionamientos, los procesos de reembolso y recuperación de boletos, así como mecanismos que garanticen la seguridad y trazabilidad de cada operación.

Con el propósito de atender estas necesidades, se desarrolló **EventFlow**, una plataforma orientada a la gestión integral de eventos que centraliza los procesos administrativos y operativos relacionados con la organización, venta de boletos y administración de asistentes. El proyecto fue concebido bajo una arquitectura moderna basada en principios de **Clean Architecture**, **Domain-Driven Design (DDD)** y el patrón **Model-View-ViewModel (MVVM)** para la aplicación móvil, permitiendo mantener una estructura escalable, desacoplada y de fácil mantenimiento.

La solución está conformada por un **backend desarrollado en Spring Boot**, encargado de la lógica de negocio, autenticación, seguridad y exposición de servicios REST, y una **aplicación móvil desarrollada en Kotlin con Jetpack Compose**, responsable de la interacción con los usuarios finales mediante una interfaz moderna y adaptable. La comunicación entre ambas capas se realiza a través de servicios REST consumidos mediante Retrofit, implementando autenticación basada en **JSON Web Tokens (JWT)** y mecanismos de autorización según los diferentes roles definidos dentro del sistema.

El presente documento tiene como finalidad recopilar de manera organizada toda la documentación correspondiente al proyecto **EventFlow**, describiendo su planificación, alcance, arquitectura, diseño, implementación y estado actual de desarrollo. Asimismo, constituye el soporte técnico y metodológico del proyecto, permitiendo comprender las decisiones tomadas durante su construcción y proporcionando una visión integral de los componentes que conforman la solución desarrollada.

Acta de Constitución del Proyecto

Objetivo del Proyecto

Desarrollar **EventFlow**, una plataforma integral para la gestión de eventos que permita administrar el ciclo completo de un evento, desde su creación hasta el control de acceso de los asistentes. La solución busca centralizar la venta de boletos, la administración de estacionamientos, el proceso de check-in mediante códigos QR dinámicos y la gestión de solicitudes de recuperación de boletos, garantizando seguridad, trazabilidad y una experiencia de usuario eficiente tanto para organizadores como para asistentes.

El proyecto se desarrolla como parte del trabajo semestral de la asignatura, aplicando principios de Ingeniería de Software, Arquitectura Limpia (Clean Architecture), Desarrollo Dirigido por el Dominio (DDD) y buenas prácticas de desarrollo de software para construir una solución escalable, mantenible y preparada para futuras ampliaciones.

Objetivos específicos

- Diseñar una arquitectura desacoplada basada en Clean Architecture y DDD.
- Implementar un sistema seguro de autenticación y autorización utilizando JWT.
- Desarrollar una aplicación móvil Android con Kotlin y Jetpack Compose para los diferentes roles del sistema.
- Construir una API REST con Spring Boot que gestione la lógica de negocio y el acceso a los datos.
- Implementar mecanismos de seguridad para evitar el fraude mediante códigos QR dinámicos.
- Gestionar la reserva y administración de estacionamientos asociados a eventos.
- Implementar el proceso de recuperación y reembolso de boletos respetando las reglas de negocio definidas.
- Garantizar la integridad y trazabilidad de todas las operaciones realizadas dentro del sistema.

Justificación

La organización de eventos involucra múltiples procesos que, en muchos casos, son administrados mediante herramientas independientes o procedimientos manuales, generando dificultades para el control de asistentes, la venta de boletos, la administración de estacionamientos y la validación del acceso al evento.

EventFlow surge como una solución tecnológica que integra estos procesos dentro de una única plataforma, permitiendo automatizar tareas administrativas, reducir errores operativos y mejorar la experiencia tanto de los organizadores como de los asistentes.

Desde el punto de vista académico, el proyecto permite aplicar de manera práctica los conocimientos adquiridos durante la carrera, integrando conceptos relacionados con:

- Ingeniería de Software.
- Arquitectura de Software.
- Desarrollo de aplicaciones móviles.

- Desarrollo Backend.
- Diseño de Bases de Datos.
- Seguridad informática.
- Desarrollo de APIs REST.
- Gestión de proyectos de software.

Asimismo, el proyecto constituye un caso de estudio completo donde se aplican metodologías modernas de desarrollo y principios de diseño utilizados en proyectos reales de la industria.

Stakeholders

Stakeholder	Rol	Interés en el proyecto
Organizador de eventos	Cliente principal	Administrar eventos, ventas, estacionamientos y asistentes.
Asistente	Usuario final	Comprar boletos, gestionar reservas, acceder al evento y consultar su información.
Administrador del sistema	Administrador	Supervisar el funcionamiento general de la plataforma y administrar parámetros globales.
Equipo de desarrollo	Desarrolladores	Diseñar, implementar y mantener la solución tecnológica.
Profesor de la asignatura	Cliente académico	Evaluar el cumplimiento de los requisitos técnicos y funcionales del proyecto.
Universidad	Institución académica	Validar la aplicación de conocimientos adquiridos durante la carrera.

Restricciones

Durante el desarrollo del proyecto se establecieron las siguientes restricciones:

Restricciones tecnológicas

- Backend desarrollado utilizando Spring Boot.
- Aplicación móvil desarrollada en Kotlin con Jetpack Compose.
- Comunicación mediante API REST.
- Base de datos PostgreSQL.
- Autenticación basada en JWT.
- Arquitectura basada en Clean Architecture y DDD.

Restricciones académicas

- Cumplir con todos los requisitos establecidos en el proyecto semestral.

- Implementar únicamente tecnologías aprobadas para la asignatura.
- Documentar adecuadamente el diseño, desarrollo y funcionamiento del sistema.
- Mantener evidencia del proceso de implementación.
-

Restricciones funcionales

- Los códigos QR deben ser dinámicos y validados por el servidor.
- Un boleto solo puede tener un propietario válido en un momento determinado.
- El sistema debe mantener la trazabilidad de todas las operaciones importantes.
- Las reglas de negocio deben ejecutarse exclusivamente desde el servidor.
- La aplicación móvil no accede directamente a la base de datos.

Restricciones de seguridad

- Protección mediante autenticación JWT.
- Validación de permisos basada en roles.
- Protección contra reutilización de códigos QR.
- Validación de operaciones críticas mediante reglas de negocio.

Supuestos

Para el desarrollo del proyecto se asumieron las siguientes condiciones:

- Los usuarios disponen de conexión a Internet para sincronizar la información con el servidor.
- Los dispositivos Android utilizados cuentan con cámara para la lectura de códigos QR.
- Los dispositivos móviles poseen sensores compatibles con Android, incluyendo acelerómetro.
- El servidor backend permanece disponible durante la operación normal del sistema.
- La infraestructura de base de datos mantiene la integridad de la información almacenada.
- Los organizadores registran correctamente la información de los eventos.
- Los asistentes proporcionan información válida durante el proceso de registro y compra.
- Las pruebas se realizan en un entorno controlado de desarrollo.

Presupuesto Académico

Al tratarse de un proyecto académico, no se contempló un presupuesto financiero formal. Sin embargo, se estimaron los recursos tecnológicos y el esfuerzo requerido para su desarrollo.

Recursos de software

Recurso	Uso
Android Studio	Desarrollo de la aplicación móvil
IntelliJ IDEA	Desarrollo del backend
PostgreSQL	Gestión de la base de datos
Postman	Pruebas de la API
Git y GitHub	Control de versiones
Docker	Contenedores para el entorno de desarrollo
Gradle	Automatización de compilación
OpenAPI / Swagger	Documentación de la API

Recursos de hardware

- Computadora personal para desarrollo.
- Dispositivo Android físico y emulador Android Studio para pruebas.
- Conexión a Internet.

Recursos humanos

Rol	Participación
Desarrolladores	Análisis, diseño, implementación, pruebas y documentación del proyecto.

Estimación del esfuerzo

Actividad	Esfuerzo estimado
Análisis y levantamiento de requisitos	40 horas
Diseño de arquitectura y base de datos	35 horas
Desarrollo Backend	120 horas
Desarrollo Android	130 horas
Integración y pruebas	60 horas
Documentación técnica	45 horas

Total estimado	430 horas
-----------------------	------------------

El presupuesto del proyecto se expresa en términos de esfuerzo académico y utilización de herramientas de software de uso libre o con licencia educativa, por lo que no representó un costo económico directo para su desarrollo.

Plan del Proyecto

Planificación General

El desarrollo de **EventFlow** se estructuró mediante una estrategia de implementación incremental, dividiendo el proyecto en módulos funcionales. Cada módulo representa un conjunto de características completas que fueron diseñadas, desarrolladas, probadas y auditadas antes de avanzar al siguiente, garantizando la estabilidad del sistema y evitando la propagación de errores entre componentes.

La planificación se basó en un enfoque iterativo inspirado en metodologías ágiles, donde cada módulo incluye actividades de análisis, diseño, implementación, pruebas y revisión de arquitectura.

Cada iteración cumple con los siguientes criterios:

- Análisis de consistencia arquitectónica.
- Implementación de funcionalidades.
- Integración con módulos existentes.
- Pruebas unitarias e integración.
- Auditoría técnica.
- Aprobación antes de iniciar el siguiente módulo.

Cronograma del Proyecto

El proyecto se desarrolló siguiendo una secuencia lógica de dependencias entre módulos, donde cada uno proporciona la base necesaria para la implementación del siguiente.

Fase	Actividad	Estado
Planificación	Definición del alcance, arquitectura y requerimientos	Completado
Diseño	Modelado de arquitectura, base de datos, API y contratos	Completado
Módulo 1	Gestión de usuarios, autenticación y seguridad	Completado
Módulo 2	Gestión de eventos y venta de boletos	Completado
Módulo 3	Sistema de estacionamientos y reservas	Completado
Módulo 4	Códigos QR dinámicos y proceso de Check-in	Completado
Módulo 5	Recuperación de boletos, reembolsos y preparación para Ticket Exchange	Completado
Integración	Integración entre módulos	Completado
Pruebas	Validación funcional e integración	Completado
Documentación	Elaboración de documentación técnica y funcional	En proceso
Entrega Final	Presentación del proyecto	Pendiente

Entregables

Durante el desarrollo del proyecto se definieron entregables parciales con el propósito de facilitar el control del avance y validar cada funcionalidad antes de su integración.

Entregables de planificación

- Documento de requerimientos.
- Arquitectura general del sistema.
- Modelo de datos.
- Diseño de API REST.
- Definición de reglas de negocio.
- Arquitectura Android.
- Arquitectura Backend.

Entregables de desarrollo

- Backend desarrollado en Spring Boot.

- Aplicación Android desarrollada con Kotlin y Jetpack Compose.
- Base de datos PostgreSQL.
- API REST documentada.
- Sistema de autenticación mediante JWT.
- Integración entre aplicación móvil y backend.

Entregables técnicos

- Scripts SQL.
- Modelo Entidad-Relación.
- Documentación OpenAPI.
- ADR (Architecture Decision Records).
- Pruebas unitarias.
- Pruebas de integración.
- Auditorías de arquitectura.

Entregables finales

- Sistema completamente funcional.
- Manual técnico.
- Manual de usuario.
- Documentación del proyecto.
- Código fuente.

Hitos del Proyecto

Los hitos representan los eventos más importantes alcanzados durante el desarrollo del proyecto.

Hito	Descripción	Estado
H1	Aprobación de la arquitectura general	Completado
H2	Finalización del diseño de la base de datos	Completado
H3	Implementación del sistema de autenticación	Completado
H4	Implementación del módulo de eventos	Completado
H5	Venta y administración de boletos	Completado
H6	Gestión de estacionamientos	Completado
H7	Implementación de QR dinámicos	Completado
H8	Implementación del proceso de Check-in	Completado
H9	Implementación del módulo de recuperación y reembolsos	Completado
H10	Auditoría técnica del Módulo 5	Completado

H11	Integración completa del sistema	Completado
H12	Documentación final del proyecto	En proceso

Roadmap del Proyecto

El desarrollo de EventFlow fue planificado mediante una hoja de ruta que permitió incorporar funcionalidades de manera progresiva, minimizando riesgos y garantizando la estabilidad del sistema.

Módulo 1 – Gestión de Usuarios y Seguridad

Objetivo

Establecer la infraestructura base del sistema.

Funcionalidades

- Registro de usuarios.
- Inicio de sesión.
- Autenticación mediante JWT.
- Control de roles.
- Refresh Token.
- Gestión básica del perfil.

Módulo 2 – Gestión de Eventos y Venta de Boletos

Objetivo

Permitir la creación y administración de eventos.

Funcionalidades

- Administración de eventos.
- Gestión de categorías.
- Venta de boletos.
- Control de disponibilidad.
- Historial de compras.
- Generación inicial de boletos.

Módulo 3 – Gestión de Estacionamientos

Objetivo

Administrar los estacionamientos asociados a cada evento.

Funcionalidades

- Creación de estacionamientos.
- Tipos de estacionamiento.
- Reservas.
- Control de disponibilidad.
- Administración de cupos.
- Integración con boletos.

Módulo 4 – QR Dinámicos y Check-in

Objetivo

Garantizar un acceso seguro al evento.

Funcionalidades

- Generación de QR dinámicos.
- Firma digital.
- Validación del QR.
- Check-in.
- Invalidación automática.
- Auditoría de accesos.

Módulo 5 – Recuperación y Reembolsos

Objetivo

Gestionar la recuperación de boletos adquiridos y preparar la infraestructura necesaria para el intercambio oficial de boletos.

Funcionalidades

- Solicitud de recuperación.
- Reembolsos.

- Integración con pagos.
- Control de elegibilidad.
- Actualización automática del inventario.
- Preparación para Ticket Exchange.
- Idempotencia.
- Auditoría de operaciones.

Dependencias entre módulos

El proyecto fue diseñado siguiendo un esquema de dependencias controladas, evitando implementaciones fuera de orden y garantizando que cada módulo utilizara únicamente funcionalidades previamente consolidadas.



Esta secuencia permitió mantener una arquitectura estable y facilitar la incorporación de nuevas funcionalidades sin afectar el comportamiento de los módulos previamente implementados.

Alcance del Proyecto

Alcance General

El proyecto **EventFlow** contempla el desarrollo de una plataforma integral para la gestión de eventos, permitiendo la administración completa del ciclo de vida de un evento, desde su creación hasta el acceso de los asistentes, incorporando mecanismos de seguridad, administración de boletos, reservas de estacionamiento y recuperación de compras.

La solución está compuesta por una aplicación móvil Android desarrollada con Kotlin y Jetpack Compose, un backend implementado con Spring Boot y una base de datos PostgreSQL.

Alcance Funcional

El proyecto incluye las siguientes funcionalidades:

- Registro e inicio de sesión de usuarios.
- Gestión de perfiles.
- Autenticación mediante JWT.
- Administración de roles.
- Creación y administración de eventos.
- Venta de boletos.
- Administración de categorías.
- Generación de códigos QR dinámicos.
- Validación de acceso mediante Check-in.
- Gestión de estacionamientos.
- Reserva de estacionamientos.
- Historial de compras.
- Gestión de pagos.
- Solicitud de recuperación de boletos.
- Procesamiento de reembolsos.
- Actualización automática del inventario.
- Integración entre aplicación móvil y backend mediante API REST.
- Consumo de servicios HTTP utilizando Retrofit.
- Persistencia local mediante Room para información de usuario y datos de apoyo.

Alcance Técnico

El proyecto contempla el desarrollo e integración de los siguientes componentes tecnológicos:

- Backend desarrollado en Spring Boot.
- API REST.
- Base de datos PostgreSQL.
- Aplicación Android.

- Arquitectura Clean Architecture.
- Patrón Repository.
- MVVM.
- Dependency Injection mediante Hilt.
- Retrofit.
- Room Database.
- Coroutines.
- JWT.
- OpenAPI.
- Auditoría de operaciones.

Exclusiones del Alcance

Para mantener el proyecto dentro de los objetivos académicos establecidos, las siguientes funcionalidades no forman parte del alcance actual:

- Integración con proveedores reales de pago (Stripe, PayPal, Yappy u otros).
- Publicación del sistema en producción.
- Portal web para clientes.
- Aplicación para iOS.
- Integración con sistemas de facturación gubernamental.
- Integración con servicios de geolocalización en tiempo real.
- Notificaciones Push mediante Firebase.
- Ticket Exchange completamente operativo (queda preparado para una fase posterior).
- Marketplace de reventa de boletos.
- Integración con sistemas externos de validación de identidad.
- Alta disponibilidad y balanceo de carga en infraestructura productiva.

Criterios de Finalización

El proyecto se considera concluido cuando:

- Todos los módulos planificados (M1–M5) han sido implementados y validados.
- Las pruebas funcionales e integrales se ejecutan satisfactoriamente.
- La aplicación Android consume correctamente los servicios REST.
- La documentación técnica y funcional se encuentra completa.

- El sistema cumple con los requisitos funcionales y no funcionales definidos para el proyecto académico.
- La entrega final incluye el código fuente, la base de datos, la documentación y los manuales correspondientes.

Requisitos del Sistema

Requisitos Funcionales

EventFlow es una plataforma integral para la gestión de eventos que permite administrar el ciclo de vida completo de un evento, desde su creación hasta el acceso del asistente mediante códigos QR dinámicos. El sistema incorpora funcionalidades para la compra de boletos, control de acceso, gestión de estacionamientos, reembolsos y recuperación de boletos, garantizando la seguridad de las transacciones y la trazabilidad de todas las operaciones.

RF-01 Gestión de Usuarios

El sistema deberá permitir:

- Registro de nuevos usuarios.
- Inicio de sesión mediante correo electrónico y contraseña.
- Recuperación de contraseña.
- Cierre de sesión.
- Actualización del perfil del usuario.
- Administración de roles (Administrador, Organizador y Asistente).

RF-02 Autenticación y Autorización

El sistema deberá:

- Autenticar usuarios mediante JWT.
- Validar los permisos asociados a cada rol.
- Renovar automáticamente el token mediante Refresh Token.
- Invalidar sesiones cerradas.

RF-03 Gestión de Eventos

El organizador podrá:

- Crear eventos.
- Modificar información del evento.
- Publicar eventos.
- Cancelar eventos.
- Configurar políticas de reembolso.
- Configurar intercambio oficial de boletos.
- Definir horarios.
- Definir capacidad.
- Configurar parqueaderos asociados.

Los asistentes podrán:

- Consultar eventos.
- Buscar eventos.
- Filtrar eventos.
- Marcar eventos como favoritos.

RF-04 Gestión de Boletos

El sistema deberá permitir:

- Crear boletos para un evento.
- Definir categorías.
- Definir precios.
- Consultar disponibilidad.
- Reservar boletos.
- Comprar boletos.
- Consultar historial de compras.

RF-05 Compra de Boletos

El sistema deberá:

- Validar disponibilidad.
- Reservar temporalmente el boleto.
- Procesar el pago.

- Confirmar la compra.
- Generar comprobante.
- Asociar el boleto al comprador.

RF-06 Gestión de Pagos

El sistema deberá:

- Registrar pagos.
- Confirmar pagos exitosos.
- Rechazar pagos fallidos.
- Registrar referencias.
- Mantener historial de transacciones.

RF-07 Gestión de QR Dinámico

El sistema deberá:

- Generar códigos QR únicos.
- Firmar digitalmente cada QR.
- Asociar el QR únicamente al propietario actual.
- Invalidar automáticamente el QR cuando cambie el propietario.
- Regenerar un nuevo QR.
- Ocultar el QR hasta el tiempo configurado previo al evento.

RF-08 Check-In

El organizador podrá:

- Escanear códigos QR.
- Validar autenticidad.
- Registrar ingreso.
- Rechazar boletos inválidos.
- Detectar códigos expirados.
- Detectar boletos previamente utilizados.

RF-09 Gestión de Estacionamientos

El organizador podrá:

- Crear estacionamientos.
- Configurar capacidad.
- Definir tipos de estacionamiento.
- Definir precios.

Los asistentes podrán:

- Reservar estacionamiento.
- Consultar disponibilidad.
- Cancelar reservas.

RF-10 Reembolsos (Refund)

El sistema deberá:

- Consultar elegibilidad.
- Calcular monto del reembolso.
- Validar políticas del evento.
- Procesar el reembolso.
- Liberar nuevamente el boleto.
- Registrar la operación en auditoría.

RF-11 Intercambio Oficial (Exchange)

El sistema deberá:

- Publicar boletos para intercambio.
- Validar elegibilidad.
- Calcular depreciación.
- Transferir la propiedad del boleto.
- Invalidar el QR anterior.
- Generar un nuevo QR.
- Registrar toda la trazabilidad.

RF-12 Recovery

El sistema deberá:

- Mostrar opciones de recuperación.
- Determinar si procede reembolso.
- Determinar si procede intercambio.
- Calcular montos correspondientes.
- Respetar las políticas configuradas para el evento.

RF-13 Gestión de Inventario

El sistema deberá:

- Controlar disponibilidad de boletos.
- Liberar boletos cancelados.
- Actualizar cupos en tiempo real.

RF-14 Notificaciones

El sistema notificará:

- Confirmación de compra.
- Confirmación de pago.
- Reembolso realizado.
- Cambio de propietario.
- Recordatorios del evento.

RF-15 Auditoría

El sistema deberá registrar:

- Compras.
- Pagos.
- Cambios de propietario.
- Reembolsos.
- Cambios de estado.
- Check-In.
- Acciones administrativas.

Requisitos No Funcionales

Seguridad

- Autenticación mediante JWT.
- Uso de Refresh Token.
- Contraseñas cifradas mediante BCrypt.
- Validación de permisos basada en roles.
- Validación de códigos QR firmados digitalmente.
- Protección contra reutilización de QR.
- Validación de integridad de las solicitudes.

Rendimiento

- Tiempo promedio de respuesta inferior a 500 ms para operaciones CRUD.
- Consultas paginadas.
- Caché local mediante Room.
- Consumo eficiente de recursos móviles.

Disponibilidad

- Disponibilidad continua del backend.
- Recuperación ante fallos.
- Persistencia de sesiones mediante Refresh Token.
- Soporte para funcionamiento parcial offline.

Escalabilidad

La arquitectura permitirá:

- Incorporar nuevos proveedores de pago.
- Incorporar nuevos métodos de autenticación.
- Escalar horizontalmente el backend.
- Agregar nuevos módulos sin afectar los existentes.

Mantenibilidad

- Clean Architecture.
- Separación de responsabilidades.
- Bajo acoplamiento.
- Alta cohesión.
- Código documentado.

Portabilidad

- Backend desplegable mediante Docker.
- Cliente Android compatible con Android 8.0 o superior.
- Base de datos PostgreSQL.

Usabilidad

- Interfaz basada en Material Design 3.
- Navegación intuitiva.
- Validaciones en tiempo real.
- Mensajes claros para el usuario.

Arquitectura del Sistema

Arquitectura General

EventFlow implementa una arquitectura basada en los principios de **Clean Architecture**, **Domain-Driven Design (DDD)** y el patrón **MVVM** para el cliente Android.

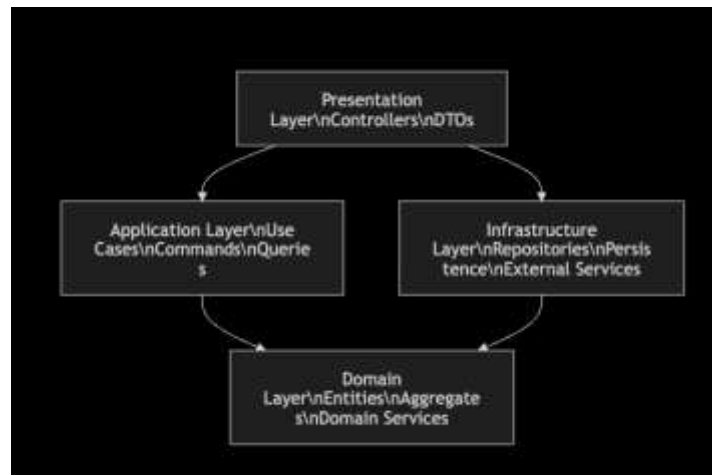
La solución se divide en dos componentes principales:

- Aplicación móvil Android.
- Backend REST desarrollado en Spring Boot.

Ambos componentes se comunican exclusivamente mediante servicios REST protegidos con autenticación JWT.

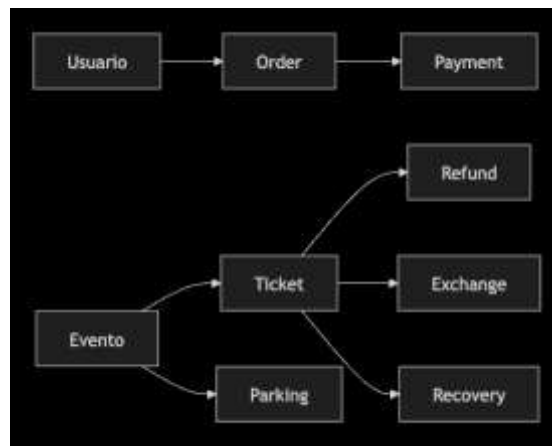
Clean Architecture

El backend está organizado en capas independientes.



Domain-Driven Design

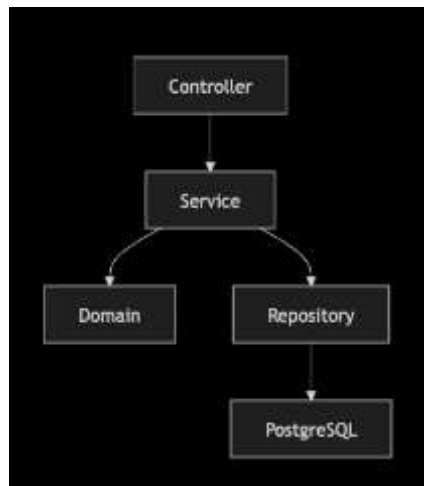
El dominio se organiza en agregados independientes.



Arquitectura Android (MVVM)



Arquitectura del Backend



Tecnologías

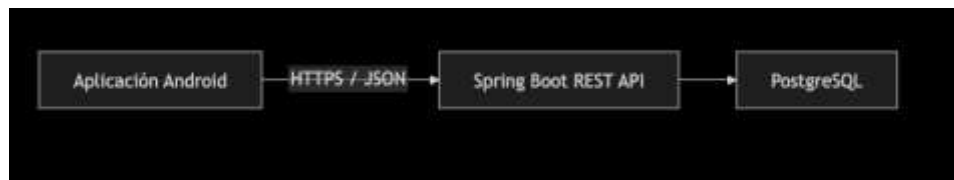
Componente	Tecnología
Cliente	Android
Lenguaje	Kotlin
UI	Jetpack Compose
Arquitectura Android	MVVM
Backend	Spring Boot
Lenguaje Backend	Java
Seguridad	Spring Security + JWT
Base de datos	PostgreSQL
ORM	Spring Data JPA
Inyección de Dependencias	Hilt
Comunicación HTTP	Retrofit
Caché Local	Room Database

Arquitectura Física

La arquitectura física está compuesta por tres capas principales:

1. Cliente Android.
2. Backend Spring Boot.
3. Base de datos PostgreSQL.

Toda la comunicación se realiza mediante HTTPS utilizando servicios REST.



Componentes Físicos

Cliente

- Aplicación Android.
- Jetpack Compose.
- Retrofit.
- Room.
- Hilt.
- ViewModel.
- Coroutines.

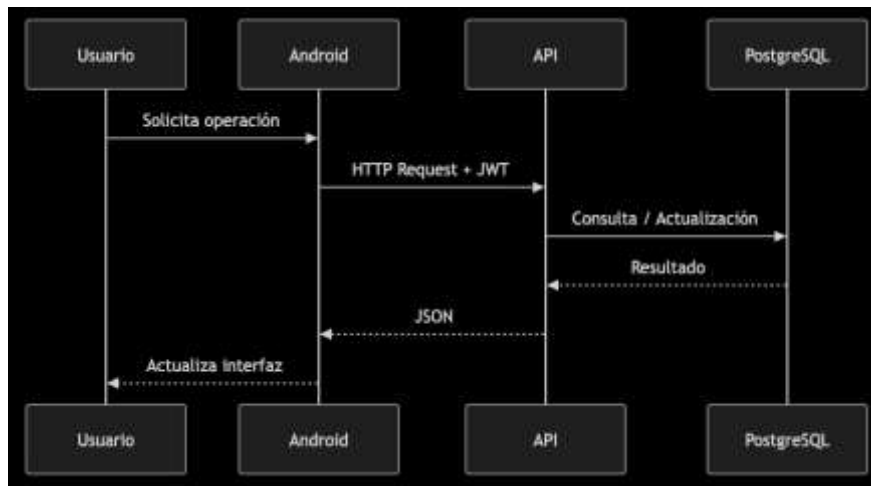
Servidor

- Spring Boot.
- Spring Security.
- Spring Data JPA.
- JWT.
- Servicios REST.
- Lógica de negocio.

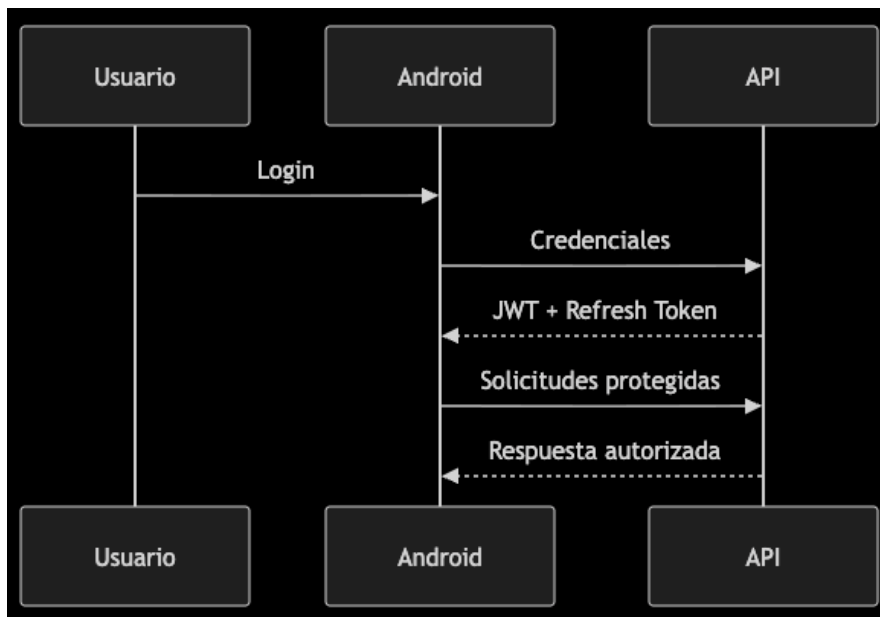
Base de Datos

- PostgreSQL.
- Persistencia de usuarios.
- Eventos.
- Boletos.
- Pagos.
- Estacionamientos.
- Reembolsos.
- Auditoría.

Flujo General de Comunicación



Autenticación

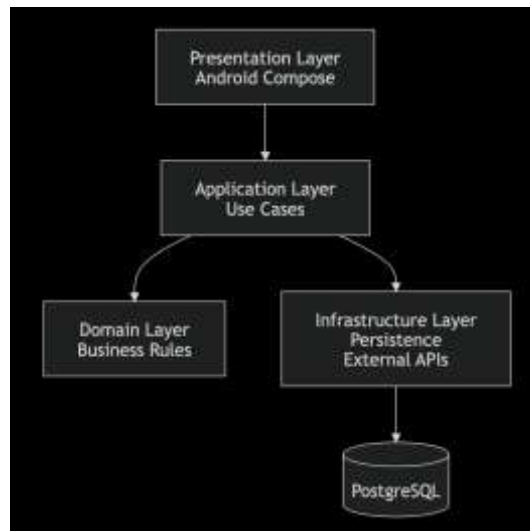


Arquitectura Lógica

La solución sigue los principios de **Clean Architecture**, complementados con **DDD (Domain Driven Design)** y el patrón **MVVM** para la aplicación Android.

Cada capa posee una responsabilidad única, minimizando el acoplamiento y favoreciendo la mantenibilidad del sistema.

Capas



Domain Layer

Esta capa representa el núcleo del negocio.

Contiene:

- Entidades
- Value Objects
- Enumeraciones
- Servicios de dominio
- Interfaces de repositorios
- Reglas de negocio

Ejemplos:

- User
- Event
- Ticket
- Order
- Refund
- Payment

En esta capa no existe dependencia de Spring Boot ni Android.

Application Layer

Implementa los casos de uso del sistema.

Responsabilidades:

- Coordinar reglas de negocio.
- Validar operaciones.
- Orquestar servicios.
- Invocar repositorios.
- Gestionar transacciones.

Ejemplos:

- CreateEventUseCase
- PurchaseTicketUseCase
- RequestRefundUseCase
- CheckInUseCase
- LoginUseCase

Infrastructure Layer

Implementa las interfaces definidas por el dominio.

Incluye:

- Spring Data JPA
- PostgreSQL
- JWT Provider

- Payment Provider
- QR Generator
- Email Service
- Repositories

Presentation Layer

Corresponde a:

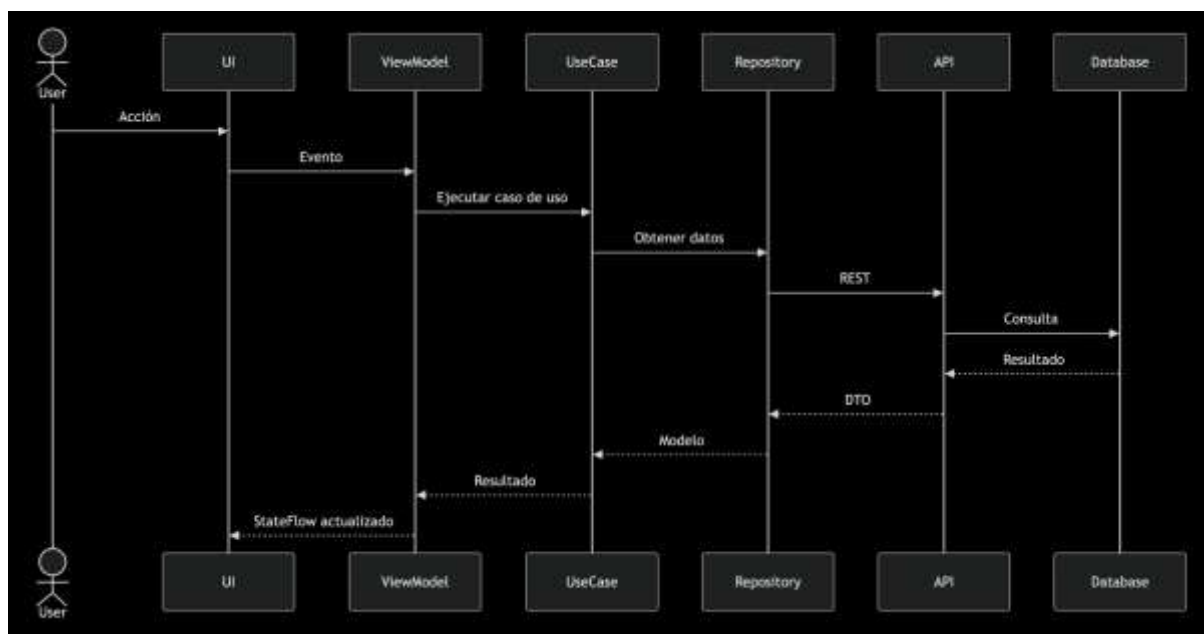
Backend:

- REST Controllers

Android:

- Jetpack Compose
- Navigation
- ViewModels

Flujo General



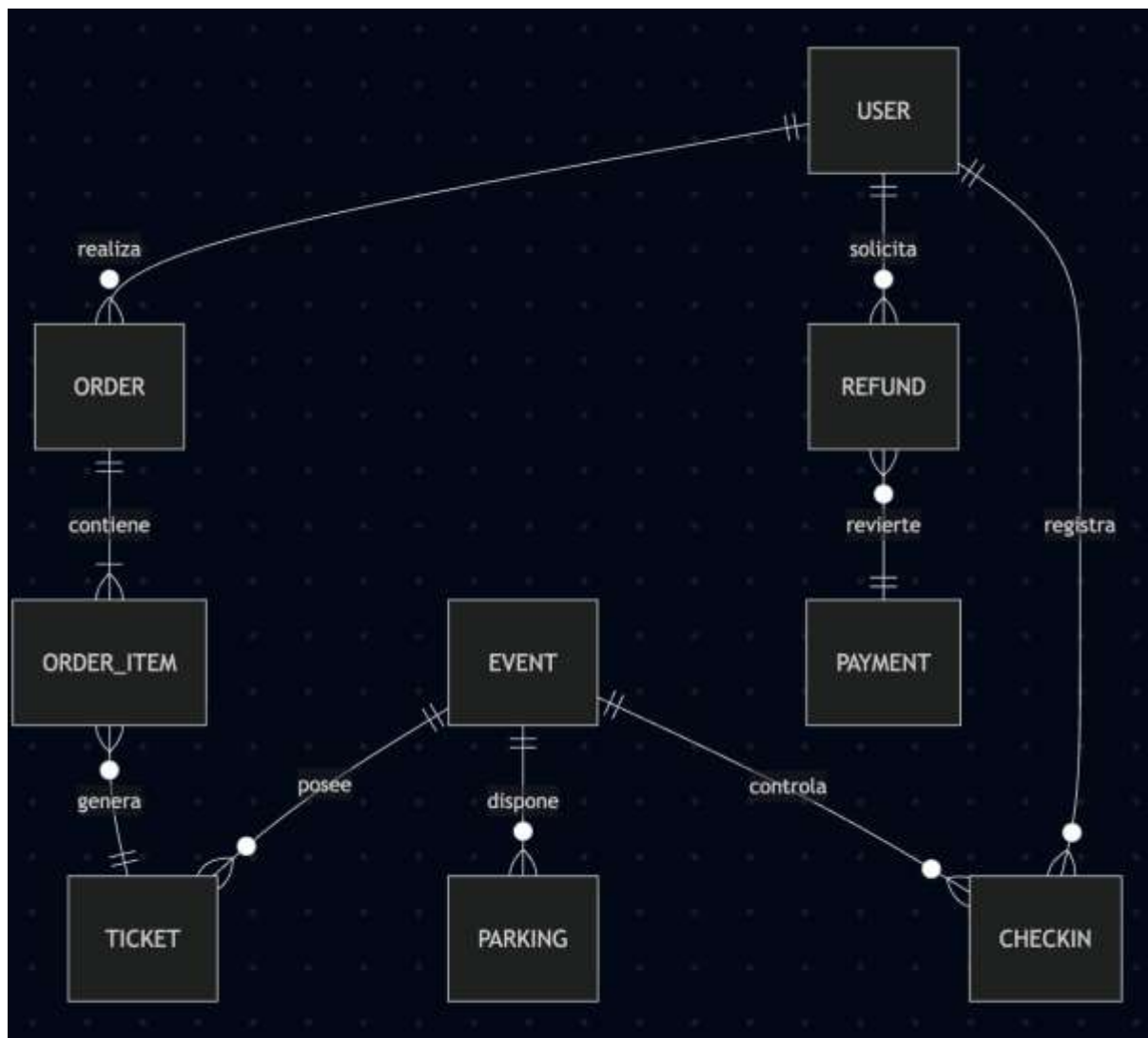
Modelo de Base de Datos

El modelo relacional fue diseñado siguiendo principios de normalización para garantizar la integridad de la información y reducir redundancias.

Entidades principales

- User
- Role
- Event
- Venue
- Ticket
- Order
- OrderItem
- Payment
- Refund
- Parking
- ParkingReservation
- Notification
- CheckIn

Relación General



Índices

Los índices implementados permiten optimizar las consultas más frecuentes.

Índice	Descripción
PK	Claves primarias de todas las tablas
FK	Relaciones entre entidades
idx_event_date	Consulta rápida de eventos
idx_ticket_owner	Consulta de boletos por usuario
idx_order_status	Consulta de órdenes por estado
uq_refunds_ticket_active	Garantiza un único reembolso activo por boleto

idx_qr_signature	Validación rápida de QR
------------------	-------------------------

OpenAPI

La API REST se encuentra organizada por módulos funcionales.

Autenticación

- POST /auth/login
- POST /auth/register
- POST /auth/refresh
- POST /auth/logout

Usuarios

- GET /users/me
- PUT /users/{id}

Eventos

- POST /events
- GET /events
- GET /events/{id}
- PUT /events/{id}

Órdenes

- POST /orders
- GET /orders
- GET /orders/{id}

Tickets

- GET /tickets
- GET /tickets/{id}
- GET /tickets/{id}/qr

Check-In

- POST /checkin

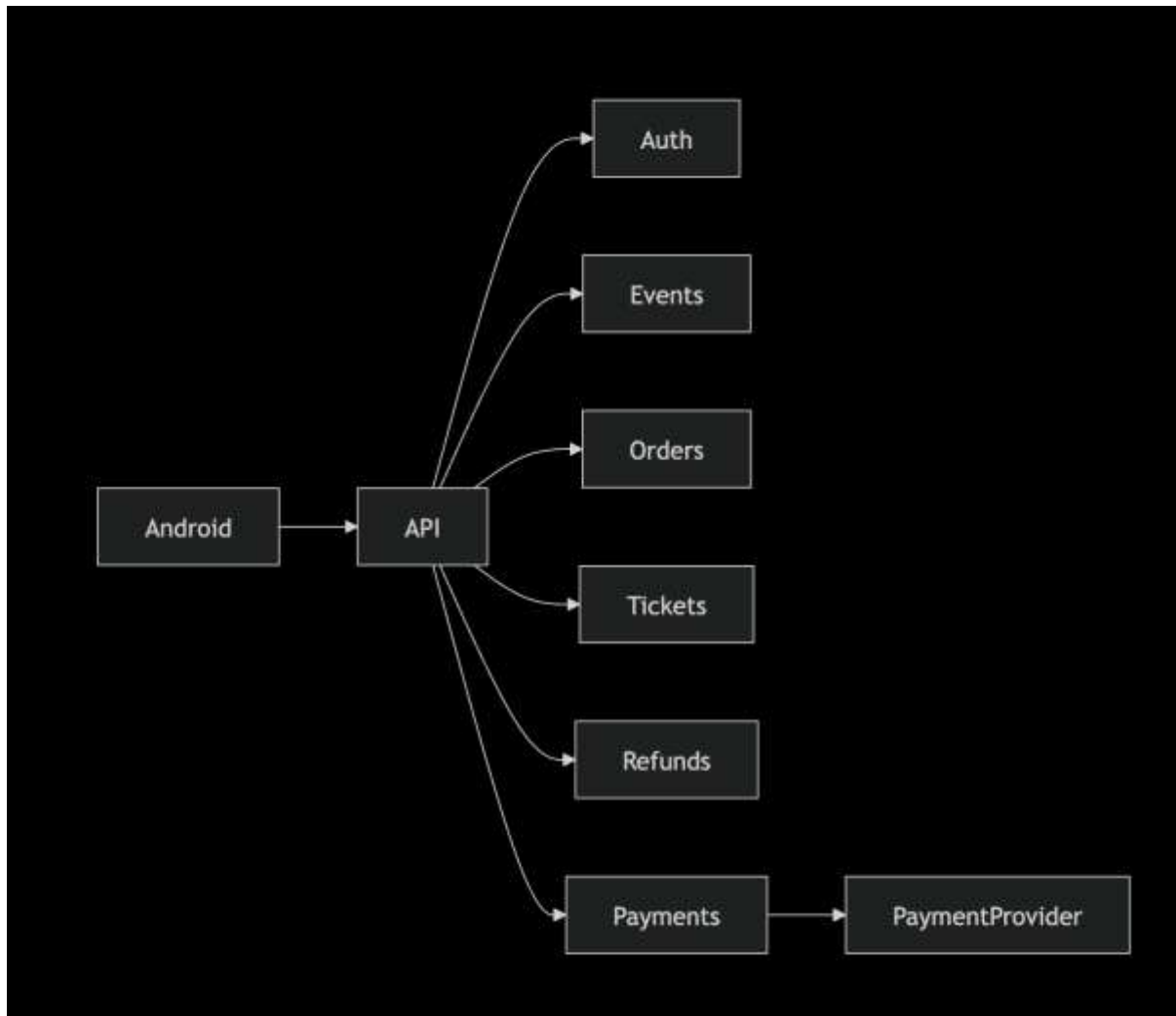
Recovery

- GET /tickets/{id}/recovery-options
- POST /refunds
- GET /refunds/{id}

Payments

- POST /payments
- POST /payments/refund

Flujo REST



. Seguridad

La seguridad se implementa bajo un modelo basado en JWT y control de acceso por roles.

Autenticación

- JWT Access Token
- Refresh Token
- Expiración configurable
- Renovación automática

Roles

Rol	Permisos
Administrador	Acceso total
Organizador	Gestión de eventos
Asistente	Compra y administración de boletos

Control de acceso

Spring Security valida:

- Token válido.
- Firma.
- Expiración.
- Rol autorizado.

QR Dinámico

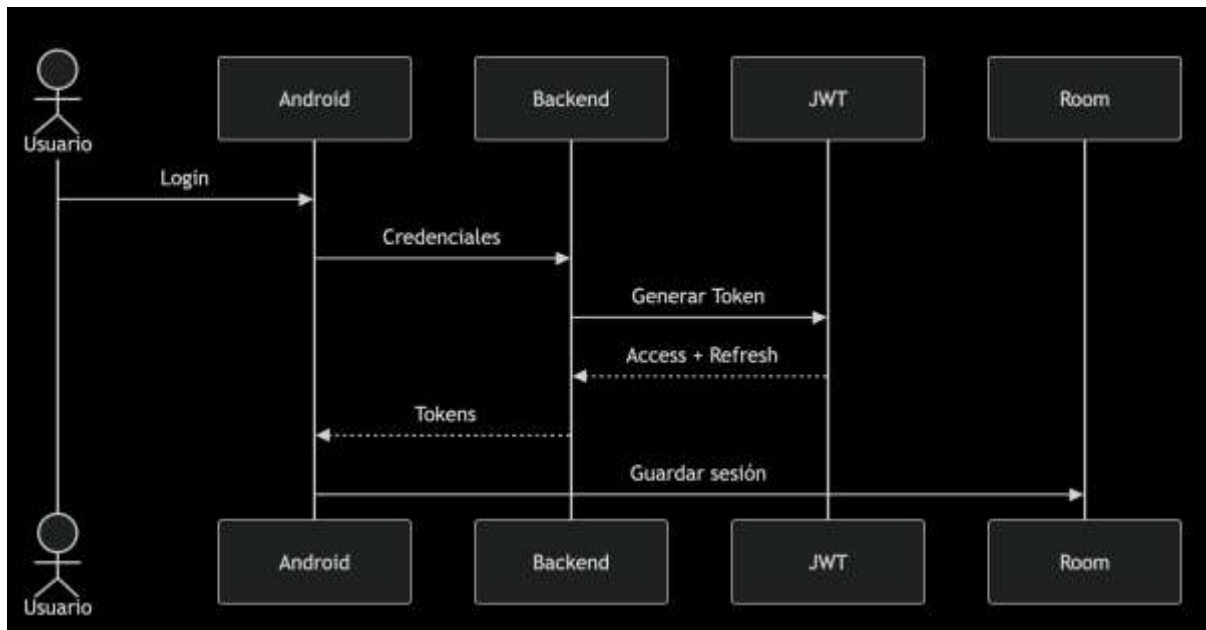
Cada boleto posee:

- identificador único
- firma digital
- expiración configurable

Cuando cambia el propietario:

- QR anterior queda inválido
- nuevo QR es generado automáticamente

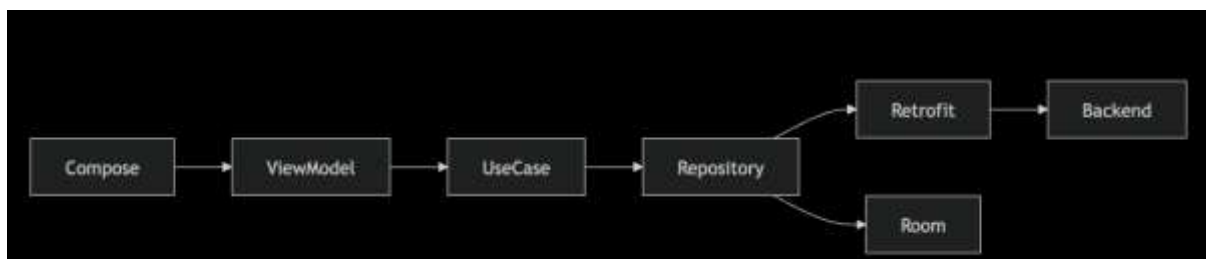
Flujo de autenticación



Diseño Android

La aplicación Android fue desarrollada utilizando **Kotlin** y **Jetpack Compose**, siguiendo el patrón **MVVM**.

Arquitectura



Jetpack Compose

Responsable de la interfaz gráfica.

Características:

- Material Design 3
- State Hoisting

- Composables reutilizables
- UI declarativa

Navigation

Se utiliza Navigation Compose para administrar la navegación entre pantallas.

Pantallas principales:

- Login
- Registro
- Home
- Eventos
- Detalle Evento
- Compra
- Tickets
- QR
- Perfil
- Órdenes

ViewModel

Responsabilidades:

- Manejo del estado.
- Comunicación con casos de uso.
- Exposición de StateFlow.

Repository Pattern

Abstrae el origen de los datos.

Puede obtener información desde:

- Backend REST
- Room
- Caché

Offline First

Cuando existe información almacenada localmente:

1. Se consulta Room.
2. Se muestra inmediatamente.
3. En segundo plano se sincroniza con el backend.
4. La UI se actualiza automáticamente mediante StateFlow.

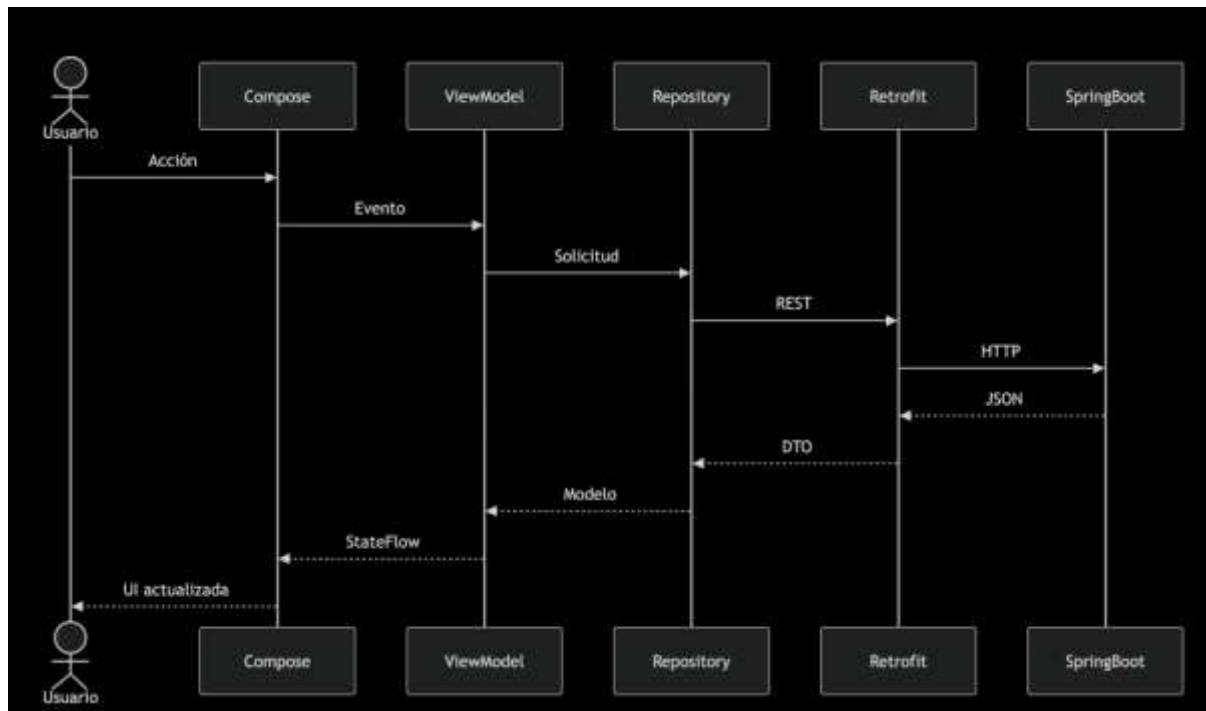
Room Database

Almacena información para funcionamiento parcial sin conexión.

Tablas principales:

- UserEntity
- TokenEntity
- EventEntity
- TicketEntity
- FavoriteEntity
- OrderEntity

Flujo Android



Gestión del Riesgo

La gestión de riesgos de EventFlow tuvo como objetivo identificar de forma temprana los eventos que pudieran afectar el alcance, tiempo, costo, calidad y seguridad del sistema, estableciendo acciones preventivas para minimizar su impacto.

Matriz de Riesgos

ID	Riesgo	Probabilidad	Impacto	Mitigación
R1	Sobreventa de boletos por concurrencia	Media	Muy Alto	Uso de transacciones ACID, SELECT FOR UPDATE, bloqueo pesimista e índices únicos.
R2	Duplicidad de QR para un mismo boleto	Baja	Muy Alto	Generación de QR dinámico, invalidación automática e índice único parcial.
R3	Compra duplicada por reintentos de red	Alta	Alto	Implementación de Idempotency-Key en todas las operaciones críticas.

R4	Fallo del proveedor de pagos	Media	Alto	Puerto PaymentProvider desacoplado y posibilidad de cambiar proveedor sin afectar el dominio.
R5	Acceso no autorizado mediante JWT robado	Baja	Muy Alto	Refresh Token rotativo, expiración corta del Access Token y revocación inmediata.
R6	Pérdida de información por eliminación accidental	Baja	Alto	Soft Delete en entidades críticas y auditoría permanente.
R7	Inconsistencia entre Android y Backend	Media	Alto	OpenAPI congelado y contrato único para ambas plataformas.
R8	Venta simultánea del mismo boleto en Exchange	Media	Muy Alto	Reservas temporales, bloqueo de fila y estado RESERVED.
R9	Errores de sincronización offline	Media	Medio	Room Database + sincronización incremental con el servidor.
R10	Lentitud del sistema por alto volumen de consultas	Media	Alto	Índices PostgreSQL, paginación y consultas optimizadas.
R11	Manipulación del QR por capturas de pantalla	Baja	Muy Alto	QR dinámico con expiración y validación exclusiva en servidor.
R12	Cambios frecuentes en requisitos	Alta	Medio	Arquitectura Hexagonal, ADRs y módulos independientes.

Evaluación de Probabilidad

Nivel	Descripción
Baja	Poco probable durante la operación normal.
Media	Puede ocurrir bajo determinadas condiciones.

Alta	Muy probable durante el ciclo de vida del proyecto.
------	---

Evaluación de Impacto

Nivel	Descripción
Bajo	No afecta funcionalidades principales.
Medio	Afecta parcialmente la experiencia del usuario.
Alto	Interrumpe funcionalidades importantes.
Muy Alto	Compromete la seguridad o integridad del sistema.

Estrategia de Mitigación

Para cada riesgo identificado se definieron medidas preventivas desde el diseño arquitectónico, evitando depender únicamente de validaciones durante la ejecución.

Las principales estrategias fueron:

- Arquitectura Hexagonal.
- Clean Architecture.
- Validaciones centralizadas.
- Auditoría mediante Outbox Pattern.
- JWT + Refresh Token.
- Idempotencia.
- Transacciones ACID.
- Concurrencia mediante bloqueos pesimistas.
- PostgreSQL como base transaccional.

Gestión del Cambio

Durante el desarrollo de EventFlow se estableció un proceso formal para controlar cualquier modificación sobre la arquitectura, reglas de negocio o base de datos.

Architectural Decision Records (ADR)

Todas las decisiones importantes fueron documentadas mediante ADR (Architectural Decision Records).

Los ADR permitieron registrar:

- problema identificado;
- alternativas evaluadas;
- decisión tomada;
- consecuencias;
- fecha de aprobación.

Entre las decisiones más importantes se encuentran:

- ADR-01 Arquitectura Hexagonal
- ADR-02 EventPolicy
- ADR-03 Snapshot de políticas
- ADR-05 Money Value Object
- ADR-06 Transacciones y concurrencia
- ADR-07 Idempotencia
- ADR-09 Outbox Pattern
- ADR-10 Expiraciones mediante Scheduler
- ADR-14 Ledger de doble partida
- ADR-15 UUID
- ADR-16 Soft Delete
- ADR-17 UTC
- ADR-19 Restricción de reembolso para boletos Exchange

Congelamiento de Arquitectura

Una vez aprobados los artefactos base, se declaró el congelamiento arquitectónico.

Los siguientes documentos no podían modificarse sin generar un nuevo ADR:

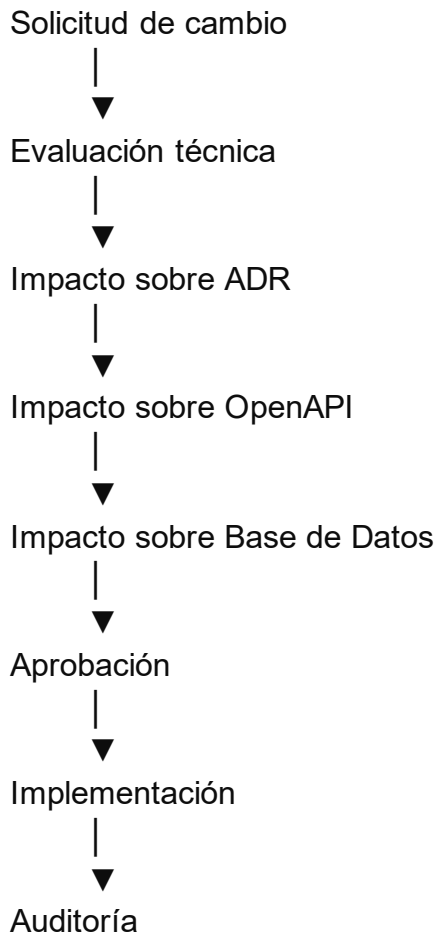
- OpenAPI
- Modelo ER
- DDL PostgreSQL
- Eventos de dominio
- Blueprint de arquitectura
- Contratos Android

- Reglas de negocio

Esto evitó inconsistencias durante los módulos posteriores.

Control de Cambios

Todo cambio siguió el siguiente flujo:



Los cambios solo fueron aprobados si:

- no rompían compatibilidad;
- mantenían las reglas del dominio;
- no afectaban la seguridad;
- respetaban la arquitectura definida.

Casos de Uso

Actores

- Visitante
- Asistente
- Organizador
- Staff de Acceso
- Administrador
- Proveedor de Pagos
- Proveedor de Notificaciones
- Scheduler
- Google Maps

Casos de uso del Visitante

- Registrarse
- Buscar eventos
- Ver eventos
- Consultar detalles
- Ver ubicación

Casos de uso del Asistente

- Iniciar sesión
- Actualizar perfil
- Comprar boletos
- Comprar parking
- Ver QR
- Consultar historial
- Guardar favoritos
- Solicitar reembolso
- Publicar boleto
- Comprar Exchange
- Unirse a Waitlist
- Aceptar oferta
- Actualizar perfil (PUT/PATCH)
- Sincronización offline

Casos de uso del Organizador

- Crear evento
- Editar evento
- Publicar evento
- Configurar políticas
- Gestionar zonas
- Gestionar parkings
- Aprobar reembolsos
- Ver estadísticas
- Invalidar boletos
- Asignar Staff

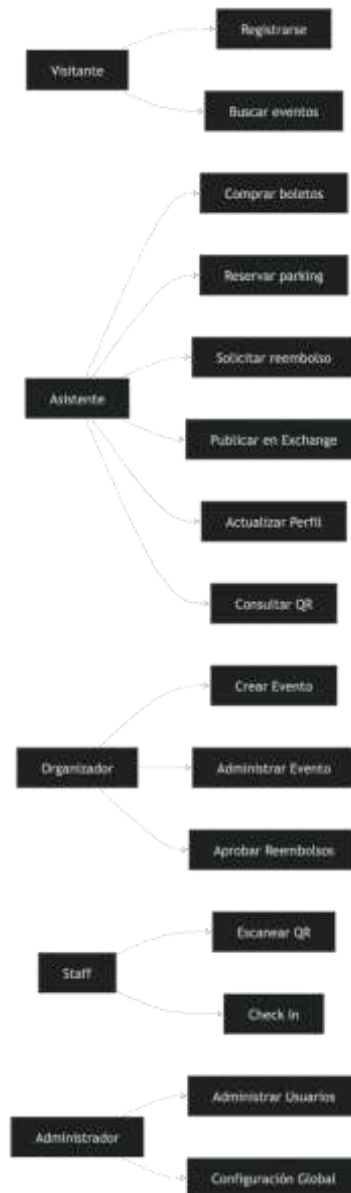
Casos de uso del Staff

- Escanear QR
- Check-in
- Check-out parking

Casos de uso del Administrador

- Administrar usuarios
- Administrar organizadores
- Configuración global
- Categorías
- Patrocinadores
- Dashboard

Diagrama de Casos de Uso



Diagramas UML

Casos de Uso

Puedes utilizar el diagrama anterior o el PlantUML que ya desarrollaste.

Diagrama de Componentes

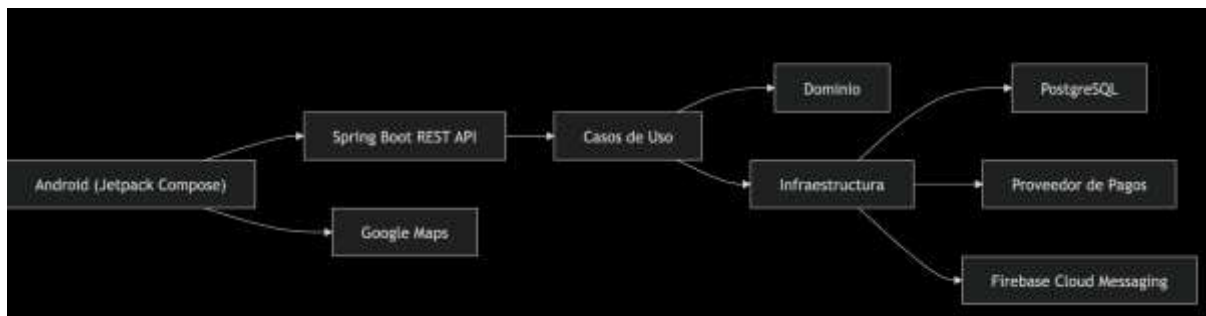


Diagrama de Clases

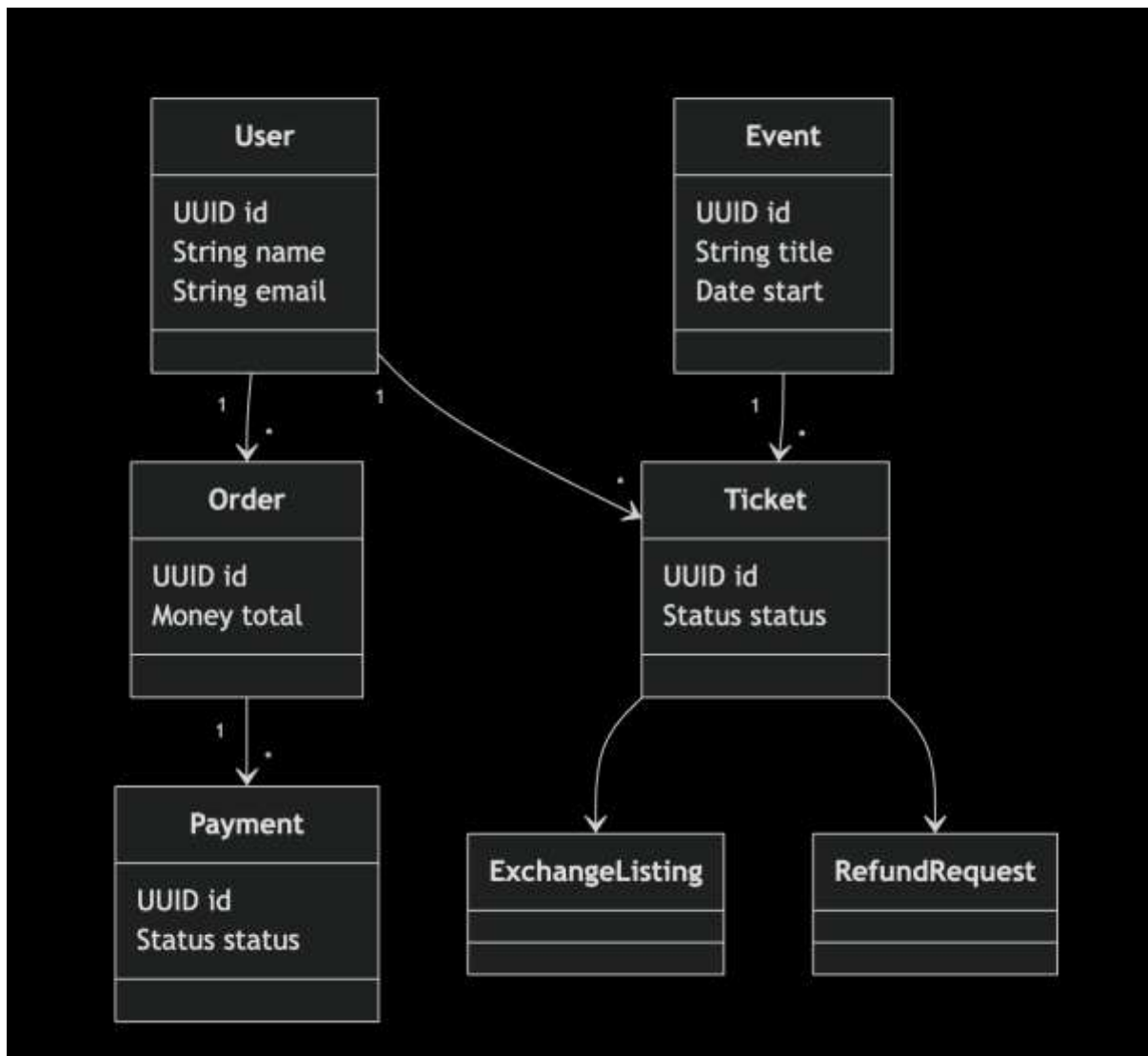


Diagrama de Despliegue

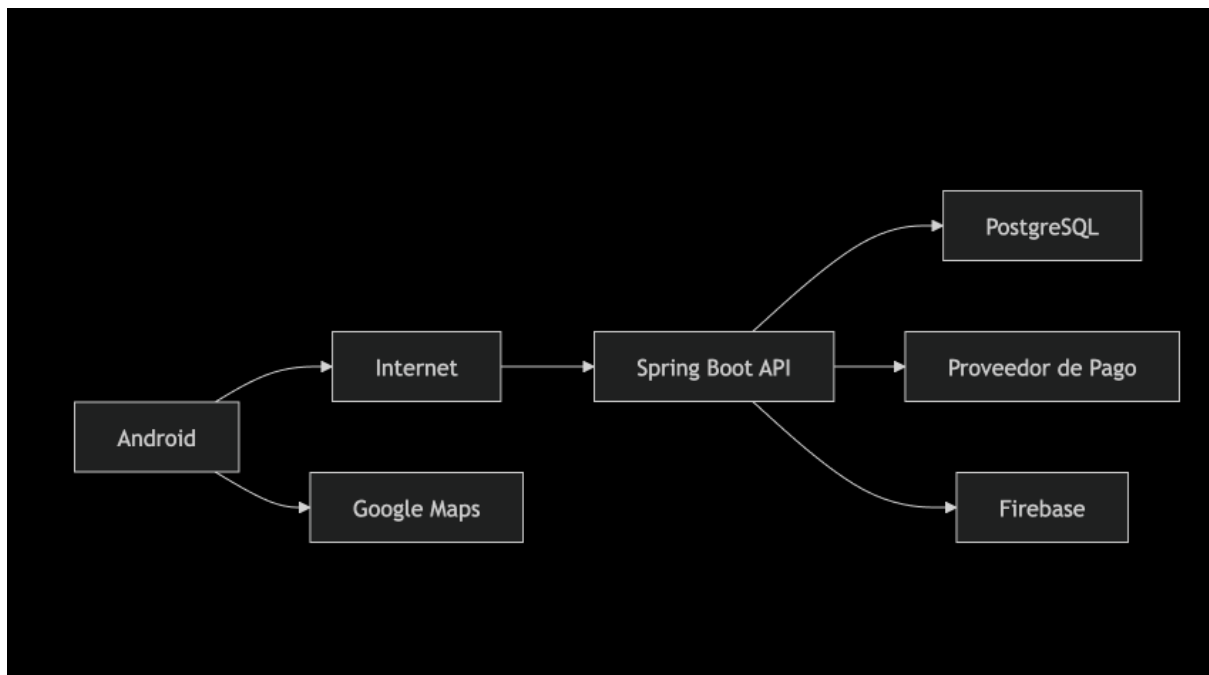
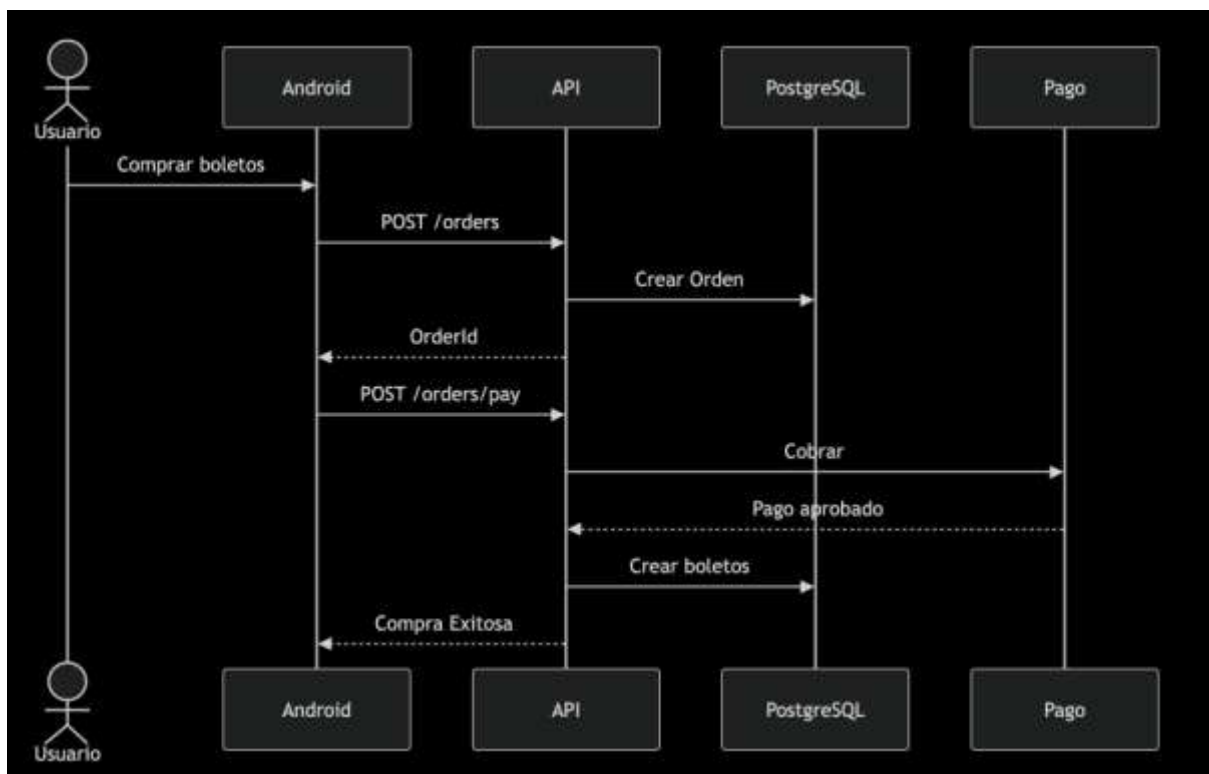


Diagrama de Secuencia (Compra)



Manual Técnico

Introducción

El presente manual técnico tiene como finalidad proporcionar las instrucciones necesarias para instalar, configurar y ejecutar correctamente el sistema **EventFlow**, tanto en el entorno de desarrollo como en un entorno de pruebas. Además, describe la arquitectura general del sistema, las tecnologías utilizadas y el procedimiento para levantar cada uno de sus componentes.

Requisitos del sistema

Para ejecutar correctamente el proyecto se recomienda el siguiente entorno de desarrollo.

Componente	Versión recomendada
Sistema Operativo	Windows 11 / Ubuntu 22.04 / macOS
Java	JDK 21
Android Studio	Koala o superior
IntelliJ IDEA	2024 o superior
PostgreSQL	16
Gradle	Compatible con AGP utilizado
Git	Última versión estable
Android SDK	API 34 o superior
Emulador Android	Android 14

Arquitectura del sistema

El sistema está compuesto por tres capas principales:

Aplicación Android

Desarrollada en Kotlin utilizando Jetpack Compose bajo la arquitectura MVVM. Es responsable de la interacción con el usuario y consume los servicios REST expuestos por el backend.

Backend

Implementado con Spring Boot siguiendo los principios de Clean Architecture y Domain Driven Design (DDD). Contiene toda la lógica de negocio, autenticación mediante JWT y exposición de la API REST.

Base de datos

PostgreSQL almacena toda la información persistente del sistema, incluyendo usuarios, eventos, boletos, estacionamientos, pagos y procesos de recuperación.

Tecnologías utilizadas

Backend

- Java 21
- Spring Boot
- Spring Security
- Spring Data JPA
- Hibernate
- JWT
- Maven

Aplicación Android

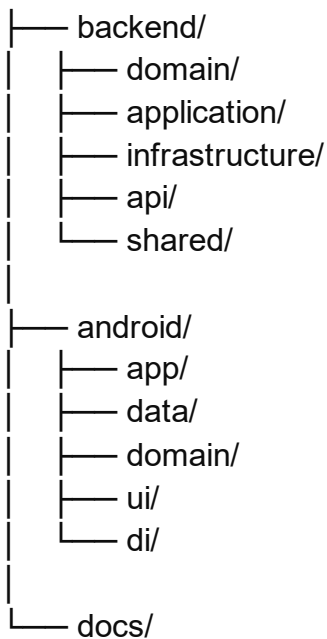
- Kotlin
- Jetpack Compose
- Navigation Compose
- ViewModel
- StateFlow
- Hilt
- Retrofit
- OkHttp
- Room
- Coroutines

Base de datos

- PostgreSQL 16

Estructura general del proyecto

eventflow/



Configuración de la base de datos

1. Instalar PostgreSQL.
2. Crear una base de datos denominada:
eventflow
3. Ejecutar el script de creación de tablas.
4. Ejecutar el script de datos iniciales.
5. Verificar que todas las tablas hayan sido creadas correctamente.

Configuración del Backend

1. Abrir el proyecto en IntelliJ IDEA.
2. Configurar el archivo de propiedades:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/eventflow
spring.datasource.username=postgres
spring.datasource.password=*****
```
3. Verificar que Maven descargue todas las dependencias.
4. Ejecutar la clase principal:
EventFlowApplication
5. Verificar que el servidor inicie correctamente en:
<http://localhost:8080>

Configuración de la aplicación Android

1. Abrir el proyecto en Android Studio.
2. Sincronizar Gradle.
3. Verificar la dirección del servidor.

<http://10.0.2.2:8080>

Esta dirección corresponde al localhost del equipo anfitrión cuando se utiliza un emulador Android.

4. Ejecutar el proyecto sobre un dispositivo físico o emulador.

Ejecución del sistema

El orden recomendado de inicio es:

1. PostgreSQL
2. Backend Spring Boot
3. Aplicación Android

Una vez iniciados los tres componentes, la aplicación estará lista para su utilización.

Consideraciones técnicas

Durante el desarrollo del proyecto se implementaron los siguientes principios:

- Clean Architecture
- SOLID
- Repository Pattern
- MVVM
- Inyección de dependencias
- JWT para autenticación
- API REST
- Persistencia local mediante Room
- Comunicación mediante Retrofit

Manual de Usuario

Introducción

El presente manual describe el funcionamiento general de la aplicación **EventFlow** desde la perspectiva del usuario final. Su objetivo es facilitar el uso de las funcionalidades implementadas durante el desarrollo del proyecto.

Inicio de sesión

Al iniciar la aplicación se presenta la pantalla de autenticación.

El usuario debe ingresar:

- Correo electrónico
- Contraseña

Posteriormente debe seleccionar **Iniciar sesión**.

Registro de usuario

Si el usuario no posee una cuenta, puede registrarse proporcionando la información solicitada por el sistema.

Una vez completado el registro podrá acceder a la aplicación utilizando sus credenciales.

Consulta de eventos

Después de autenticarse, el usuario podrá visualizar el listado de eventos disponibles.

Cada evento muestra información relevante como:

- Nombre
- Fecha
- Ubicación
- Precio
- Capacidad disponible



Imagen. Pantalla principal de eventos

Consulta del detalle del evento

Al seleccionar un evento se presenta información detallada acerca del mismo.

Entre la información disponible se encuentra:

- Descripción
- Fecha
- Ubicación

- Tipos de boletos
- Disponibilidad
- Información adicional

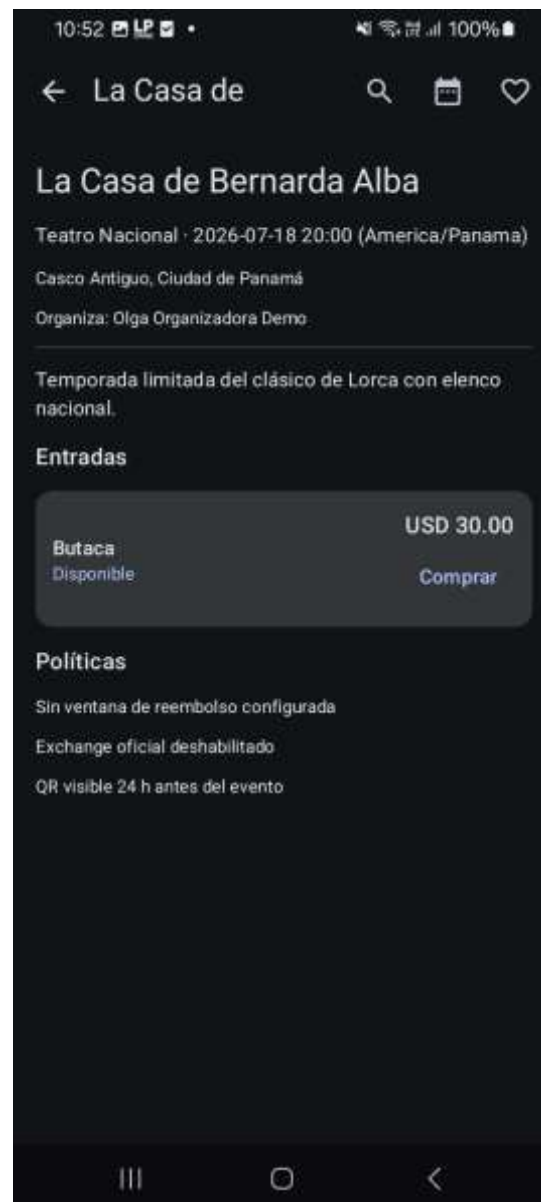


Imagen. Pantalla de detalle del evento

Compra de boletos

El usuario puede seleccionar el tipo y cantidad de boletos disponibles.

Posteriormente confirma la compra siguiendo el flujo establecido por el sistema.

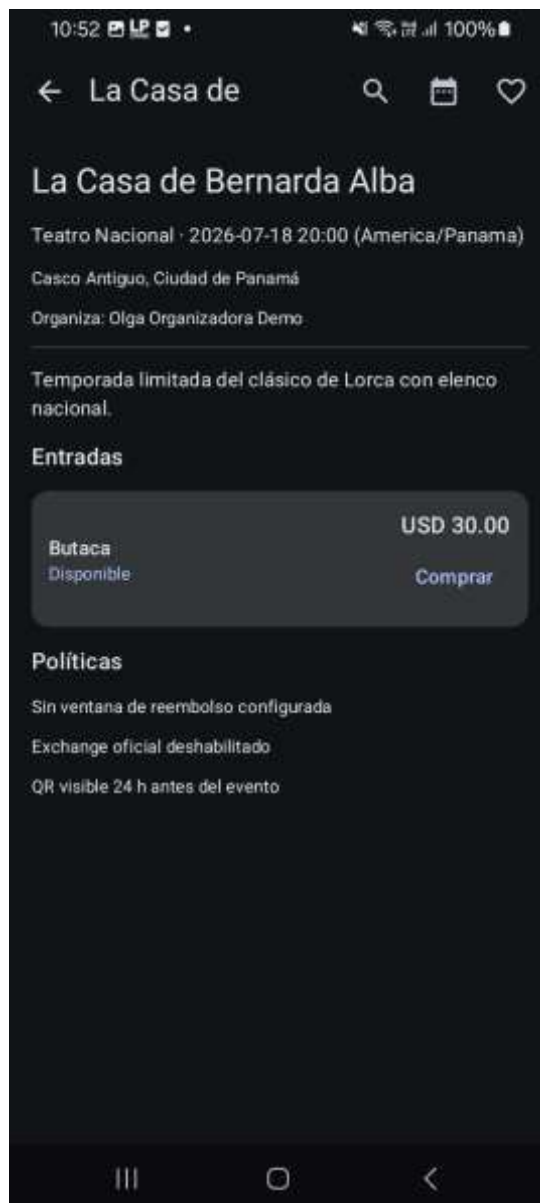


Imagen. Pantalla de compra.

Consulta de boletos

Una vez finalizada la compra, el usuario podrá consultar todos los boletos adquiridos desde la sección correspondiente.

Cada boleto muestra:

- Código
- Estado
- Evento

- Fecha



Imagen. Pantalla de boletos.

Código QR

Cada boleto posee un código QR dinámico utilizado durante el proceso de ingreso al evento.

Este código únicamente estará disponible cuando se cumplan las reglas definidas por el sistema.

Consulta de órdenes

El usuario puede consultar todas las órdenes realizadas.

La información incluye:

- Número de orden
- Fecha
- Estado
- Total pagado

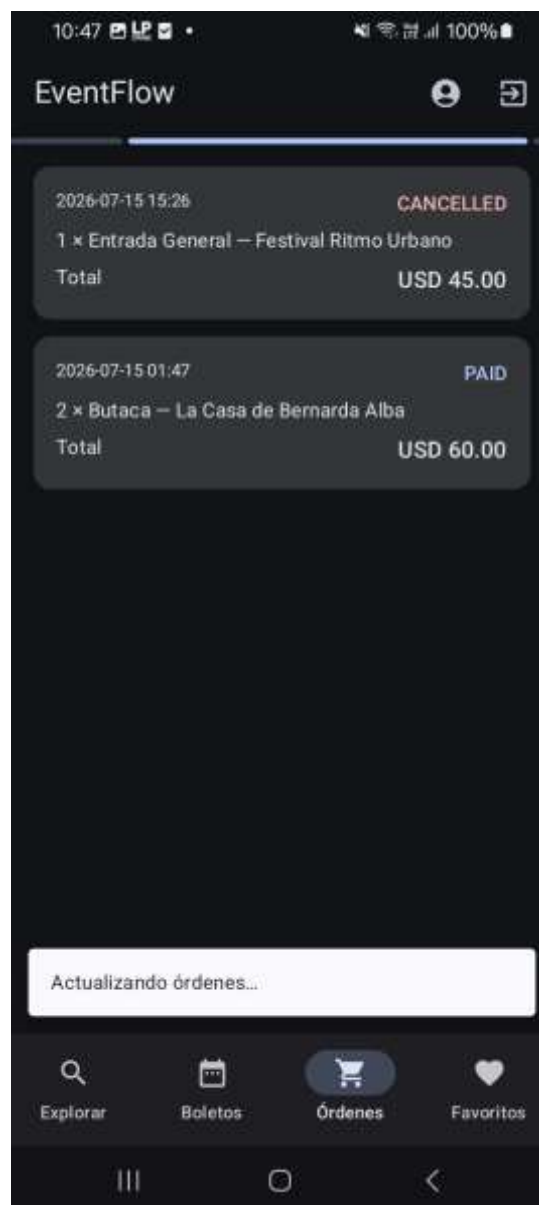


Imagen. Pantalla de órdenes.

Actualización automática mediante acelerómetro

Como parte de los requisitos académicos implementados, la aplicación utiliza el acelerómetro del dispositivo.

Cuando el usuario se encuentra en la pantalla de órdenes y realiza un movimiento de sacudida (Shake), la aplicación actualiza automáticamente la información consumiendo nuevamente el servicio REST correspondiente.

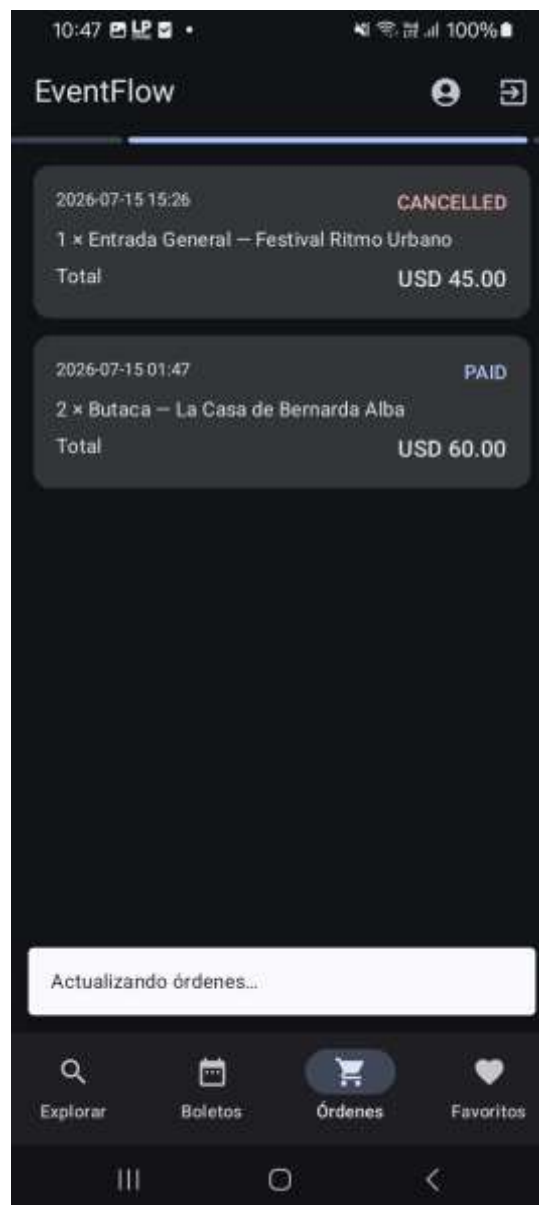


Imagen. Pantalla de órdenes antes y después de la actualización.

Actualización del perfil

El usuario puede modificar la información editable de su perfil utilizando la opción correspondiente.

Los cambios son enviados al servidor mediante una solicitud HTTP PUT y se reflejan inmediatamente en la aplicación.

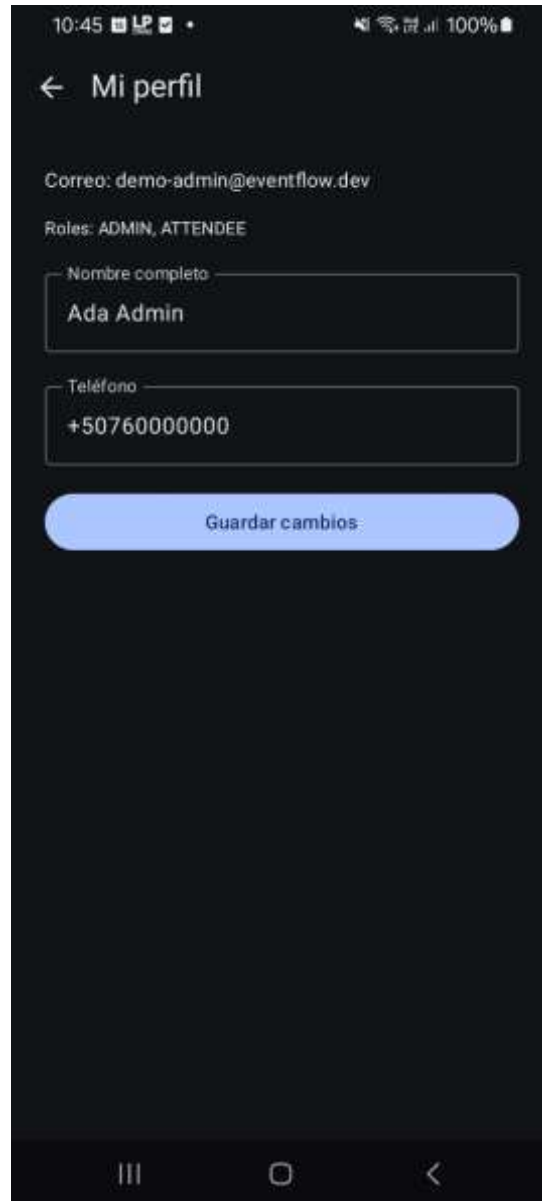


Imagen. Pantalla de edición del perfil.

Cierre de sesión

Para finalizar la sesión el usuario únicamente debe seleccionar la opción **Cerrar sesión**, eliminando el token de autenticación almacenado localmente.

Conclusiones

El desarrollo del proyecto **EventFlow** permitió aplicar de manera integral los conocimientos adquiridos durante la formación en Ingeniería de Software, combinando conceptos de arquitectura de software, desarrollo móvil, desarrollo backend, bases de datos y gestión de proyectos.

Durante el proceso de desarrollo se implementó una arquitectura basada en **Clean Architecture** y **Domain Driven Design (DDD)**, permitiendo una clara separación de responsabilidades entre las distintas capas del sistema y facilitando la mantenibilidad y escalabilidad del software.

La utilización de tecnologías modernas como **Spring Boot**, **PostgreSQL**, **Kotlin**, **Jetpack Compose**, **Retrofit** y **Room** permitió construir una solución robusta, modular y preparada para futuras ampliaciones, manteniendo buenas prácticas de desarrollo y un alto nivel de organización del código.

Asimismo, el proyecto incorporó mecanismos de seguridad mediante autenticación basada en **JWT**, comunicación mediante servicios REST, persistencia local para funcionamiento parcial sin conexión y un conjunto de reglas de negocio que garantizan la integridad de la información durante los procesos de compra, gestión de boletos y recuperación.

Desde la perspectiva académica, el proyecto permitió fortalecer competencias relacionadas con el análisis, diseño, implementación, documentación y validación de sistemas de información, integrando tanto aspectos técnicos como metodologías de desarrollo y gestión de proyectos.

Finalmente, el sistema desarrollado constituye una base sólida para la incorporación de nuevas funcionalidades en futuras versiones, como la implementación completa del intercambio oficial de boletos, integración con proveedores de pago reales, notificaciones en tiempo real y mejoras en los procesos de administración y monitoreo de eventos, manteniendo siempre una arquitectura preparada para evolucionar sin afectar los componentes existentes.